빅 데이터 분석을 위한 알고리즘 활용

빅 데이터분석을 위한 알고리즘 소개 및 Python을 활용한 실습

SUNGHEE KUK

DATA Analysis

2025
프로그래밍

CONTENTS

01 파이썬 프로그래밍 기초

파이썬에서 활용 가능한 자료의 형태와 기능들에 대해 학습함

02 데이터 수집

데이터 수집과정을 소개하고 파이썬을 이용하여 데이터를 수집하는 방법을 학습함

03 데이터 전처리 및 시각화

분석 데이터의 구조 및 형태와 파이썬을 활용하여 분석 데이터를 생성하기 위한 방법들을 학습함

04 AI 알고리즘 맛보기

회귀모형과, Tree기반의 앙상블 모형에 대해 간단히 학습하고 파이썬으로 생성함

05 모형 평가 방법

다양한 AI 알고리즘들을 생성하고 평가하는 방법에 대해 학습함

5.1 모형 평가 방법론 소개

- 여러가지의 예측 알고리즘을 생성한 후, 최종 모형을 선택하기 위해서는 생성한 모형이 얼마나 좋은 성능을 보이는지 수치적으로 비교할 수 있는 평가 지표가 필요합니다.
- 모형 평가를 위한 지표는 수치 예측 문제와 분류 예측 문제에 따라 다른 지표를 사용합니다.

수치 예측 문제인 경우

MSE (Mean Squared Error)

- 실제 값과 예측 값의 차이를 제곱한 값의 평균을 구한 지표
- 대표적으로 사용되는 지표로 계산 및 이해하기가 쉽다는 장점을 가지고 있으나 타겟의 스케일에 의존적임
- 에러를 제곱하면서 발생하는 왜곡을 보완하기 위해 루트를 사용한 RMSE(Root Mean Squared Error)도 많이 사용됨

$$MSE = \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{n}, \qquad RMSE = \sqrt{\frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{n}}$$

MAPE (Mean Absolute Percentage Error)

- 실제 값 대비 실제 값과 예측 값의 차이에 대한 절대값 비율의 평균을 퍼센트로 표현한 지표
- 타겟의 스케일에 대한 문제를 보완하고 직관적이지만 타겟의 값이 매우 작은 경우 무한대에 가까운 값이 생성될 수 있음

$$MAPE = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|, \qquad MPE = \frac{100}{n}\sum_{i=1}^{n}\frac{y_i - \widehat{y_i}}{y_i}$$

분류 예측 문제인 경우

Accuracy / Precision / Recall

- Accuracy(정확도)는 실제 값에서 예측 값이 얼마나 같은지를 판단하는 지표로 전체 예측된 데이터의 건수에서 실제 값과 동일한 데이터 건수의 비율을 나타낸 지표임
- Precision(정밀도)와 Recall(재현율)은 예측 값 중에서도 대상이 되는 예측 값("1")에 초점을 둔 평가 지표임

예측 값

실제 값	0	1
0	445	15
1	35	5

Confusion matrix

정확도 : 450/500 = 0.90
정밀도 : 5/20 = 0.25
재현율 : 5/40 = 0.125

F1 Score

- Precision과 Recall을 결합한 지표

$$F1 = \frac{1}{\frac{1}{recall} + \frac{1}{precision}} = 2 \times \frac{precision \times recall}{precision + recall}$$

5.1 모형 평가 방법론 소개

- 정밀도와 재현율은 서로 상호 보완적인 지표이기 때문에 양쪽 모두를 올릴 수는 없는데 이러한 관계를 Precision-Recall Trade-Off라고 합니다.
- AUC 스코어는 분류 문제에서 모형의 성능을 평가하는 지표들 중 하나로 ROC 곡선에 기반한 면적의 값을 의미합니다.

Trade-Off

Precision-Recall Trade-Off

- 분류 문제의 특성상 예측 결과에 대한 확률 값을 도출한 경우 분류를 하기 위해 특정 확률을 기준으로 1과 0을 나눠주는 임계값(Threshold)이 필요함
- Precision과 Recall은 이 임계값을 통해 조정될 수 있으나 두 지표는 서로 상호 보완적인 지표이기 때문에 한 쪽을 올리는 경우 다른 한 쪽이 줄어드는 현상이 발생되는데 이를 Precision-Recall Trade-Off라고 함

Threshold = 0.5

	0	1
0	343	14
1	30	182

정확도 : 525/569 = 0.92

정밀도 : 182/196 = 0.93

재현율 : 182/212 = 0.86

Threshold = 0.8

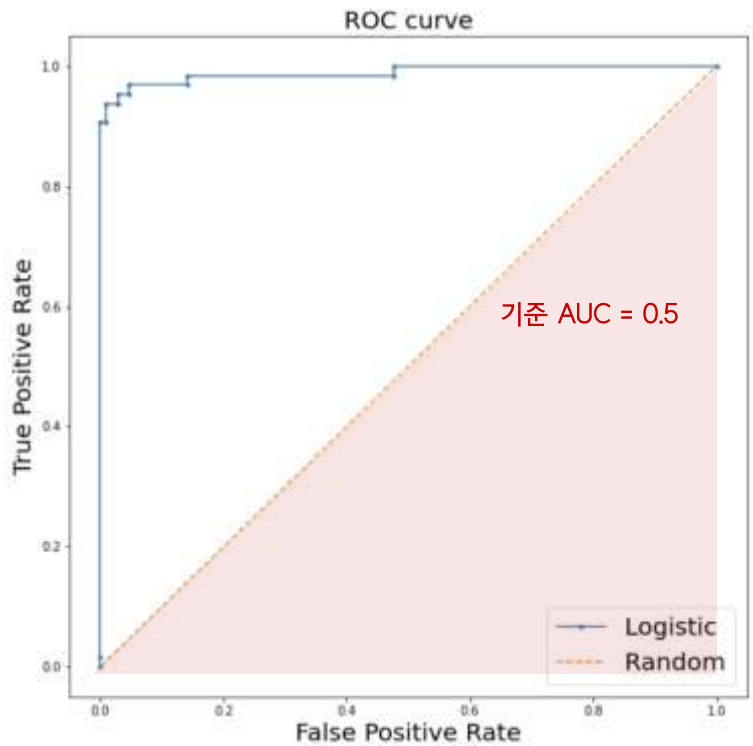
	0	1
0	352	5
1	54	158

정확도 : 510/569 = 0.90

정밀도 : 158/163 = 0.97

재현율 : 158/212 = 0.75

AUC Score



5.2 Overfitting

- 생성한 분석 모형이 분석 데이터에 과도하게 맞춰져서 생성되어 모형이 굉장히 복잡하게 되었을 때 주로 나타나는 현상으로 분석 데이터 외의 새로운 데이터에서 예측 성능이 크게 저하되는 경우를 Overfitting 문제라고 합니다.
- Overfitting 문제를 방지하기 위해 모형을 생성하는 경우 모든 데이터를 사용하는 것이 아닌 모형 생성을 위한 훈련 데이터와 이를 검증하기 위한 테스트 데이터로 나누어서 사용되어야 합니다.

Overfitting

Overfitting (과대 적합)

- 분석 데이터에만 존재하는 특징에 과대하게 학습되어 작은 변화에도 과장된 결과를 초래하여 새로운 데이터에 대한 예측력이 떨어지는 문제를 의미함

발생 원인

- 모형을 생성할 때 모든 데이터를 사용한 경우
- 설명 변수가 너무 많이 존재하여 생성한 모형이 복잡한 경우

가능한 다양한 정보가 포함되어 있는 데이터로 모델링 해야함

Overfitting 방지 방법

훈련 데이터 / 테스트 데이터

- 데이터를 모형 생성에 직접적으로 사용되는 훈련 데이터와 테스트 데이터로 나누어서 사용함으로써 모형을 생성하는 경우는 훈련 데이터만을 사용하고 해당 모형의 성능을 평가하는 경우 테스트 데이터를 활용하여 모형을 비교함
- 보통 훈련 데이터와 테스트 데이터의 비율을 7:3이나 8:2의 비율로 사용함

Cross Validation

- 데이터를 분할하여 모형 생성 및 적용에 번갈아 사용함으로써 모형을 일반화 하여 평가하고자 하는 방법으로 보통 데이터를 10개의 등분으로 나누어서 사용하는 10-fold cross validation 방법이 대표적임

5.3 실습

- 랜덤 포레스트(Random Forest)는 의사 결정 나무(Decision Tree)에서의 단점인 과적합(overfitting)문제를 보완하는 방법으로 높은 예측력을 보인다는 장점을 가지고 있습니다.
- 그러나 의사 결정 나무에 비해 여러 개의 나무를 사용하기 때문에 해석의 어려움이 있습니다.

실습 데이터

데이터 설명

- 위스콘신 유방암 데이터는 종양의 크기, 모양 등의 다양한 속성 값을 기반으로 해당 종양이 악성인지 양성인지를 분류한 데이터임
- 총 32개의 column과 569개의 row를 가지고 있음

- **radius** : 중심에서 외벽까지 거리
- **texture** : 질감
- **area** : 면적
- **perimeter** : 둘레
- **smoothness** : 매끄러운 정도
- **compactness** : 조그만 정도
- **concavity** : 오목함
- **points** : 오목한 점의 수
- **symmetry** : 대칭 정도
- **dimension** : 프랙탈 차원

평균값,
표준 오차값,
제일 큰 3개의 값을 평균낸 값

- **diagnosis** : 양성 여부 (1, 양성 : 0, 악성)

	mean radius	mean texture	mean area	mean symmetry	DIAG RES
0	17.99	10.38	1001.0	0.2419	0
1	20.57	17.77	1326.0	0.1812	0
2	19.69	21.25	1203.0	0.2069	0
3	11.42	20.38	386.1	0.2597	0
4	20.29	14.34	1297.0	0.1809	0

실습 단계 설명

최종 결과 확인 데이터

- 569개의 데이터에서 처음 50개의 row를 결과 비교를 위해 최종 결과 확인용 데이터로 사용함

분석 데이터

- 569개의 데이터에서 처음 50개의 row를 제외한 519개의 row를 활용하여 비교함

비교 방법

- 전체 데이터 활용 : 이전 실습에서 해본 것 처럼 519개의 모든 row를 활용하여 분석 모형 생성하고 가장 좋은 결과를 가진 모형을 선택함
- Train / Test 활용 : Train data와 Test data를 나누어 훈련용 데이터를 통해 모형을 생성한 후 Test 데이터를 기준으로 가장 좋은 모형을 선택함 (8:2의 비율을 사용함)

5.3 실습

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from os import system  # Tree 시각화를 위함
system("pip install graphviz")
import graphviz
```

```
0
```

```
# 유방암 데이터 로드
cancer = load_breast_cancer()

# 유방암 데이터 DataFrame으로 변환
cancerDF = pd.DataFrame(cancer.data, columns = cancer.feature_names)
```

```
# 유방암 데이터의 타겟 변수와 정의 (1이면 양성 종양, 0이면 악성 종양)
cancerDF['diagnosis'] = cancer.target
# 보통 1을 주요 타겟이 되는 범주로 정의함
cancerDF['diagnosis'] = np.where(cancerDF['diagnosis'] == 0, 1, 0)
cancerDF.shape
```

```
(569, 31)
```

```
# 최종 테스트 데이터로 사용할 50개의 관측값 분류
final_testDF = cancerDF.loc[0:49]
final_testDF.head()

cancerDF = cancerDF.loc[50:]
cancerDF.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean c
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

5 rows × 31 columns

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean
50	11.76	21.60	74.72	427.9	0.08637	
51	13.64	16.34	87.21	571.8	0.07685	
52	11.94	18.24	75.71	437.6	0.08261	
53	18.22	18.70	120.30	1033.0	0.11480	
54	15.10	22.02	97.26	712.8	0.09056	

모든 데이터를 활용하여 모형을 생성

1.1.로지스틱 회귀분석 (Logistic Regression)

```
[51] from sklearn.linear_model import LogisticRegression
     from sklearn.preprocessing import StandardScaler

     # 타겟 변수 설명 변수 데이터 정의
     y_target = cancerDF['diagnosis']
     X_data   = cancerDF.drop('diagnosis', axis = 1)

     # 표준화 작업
     scaler = StandardScaler()
     scaler.fit(X_data)
     X_scale = pd.DataFrame(scaler.transform(X_data), columns = X_data.columns)

     # 로지스틱 회귀 모형 생성
     Logistic = LogisticRegression()
     Logistic.fit(X_scale, y_target)

     # 예측 확률
     probs = Logistic.predict_proba(X_scale)[:,1]

     # 타겟 변수 예측
     pred_logistc = np.where(probs.reshape(-1) >= 0.5, 1, 0)
```

```
StandardScaler()LogisticRegression()
```

1.2. 의사 결정 나무(Decision Tree)

```
from sklearn import tree

# 의사결정나무(Decision Tree) 모형 생성
DTree = tree.DecisionTreeClassifier(random_state=300)
DTree.fit(X_data, y_target)

# 의사결정나무(Decision Tree) 모형 생성 (depth를 3으로 준것)
DTree3 = tree.DecisionTreeClassifier(random_state=300, max_depth = 3)
DTree3.fit(X_data, y_target)
```

```
DecisionTreeClassifier(random_state=300)DecisionTreeClassifier(max_depth=3, random_state=300)
```

```
[58] # 타겟 변수 예측 (depth를 주지 않은 것과, 준 것)
     pred_Tree  = DTree.predict(X_data)
     pred_Tree3 = DTree_depth3.predict(X_data)
```

1.3. 랜덤 포레스트 (Random Forest)

```
[45] from sklearn.ensemble import RandomForestClassifier

     # 100개의 decision tree를 사용한 랜덤 포레스트 (max_features는 sqrt개수를 사용하는 것이 디폴트임)
     RF = RandomForestClassifier(n_estimators=100, random_state=0)
     RF.fit(X_data, y_target)

     # 예측값 생성
     pred_RF = RF.predict(X_data)
```

모든 데이터를 활용한 모형 평가

```
print("Logistic 결과")
pd.DataFrame(confusion_matrix(y_target, pred_logistc))

print("Logistic 모형 평가")
print(classification_report(y_target, pred_logistc,digits=4))
```

```
Logistic 결과
     0   1
0  350   0
1    5 164
Logistic 모형 평가
              precision    recall  f1-score   support

           0     0.9859    1.0000    0.9929       350
           1     1.0000    0.9704    0.9850       169

    accuracy                         0.9904       519
   macro avg     0.9930    0.9852    0.9889       519
weighted avg     0.9905    0.9904    0.9903       519
```

```
[49] print("Depth를 3으로 제한한 의사결정 나무 결과")
pd.DataFrame(confusion_matrix(y_target, pred_Tree3))

print("Depth를 3으로 제한한 의사결정 나무 모형 평가")
print(classification_report(y_target, pred_Tree3,digits=4))
```

```
Depth를 3으로 제한한 의사결정 나무 결과
     0   1
0  349   1
1   18 151
Depth를 3으로 제한한 의사결정 나무 모형 평가
              precision    recall  f1-score   support

           0     0.9510    0.9971    0.9735       350
           1     0.9934    0.8935    0.9408       169

    accuracy                         0.9634       519
   macro avg     0.9722    0.9453    0.9572       519
weighted avg     0.9648    0.9634    0.9629       519
```

```
[48] print("Depth 제한 없는 의사결정 나무 결과")
pd.DataFrame(confusion_matrix(y_target, pred_Tree))

print("Depth 제한 없는 의사결정 나무 모형 평가")
print(classification_report(y_target, pred_Tree,digits=4))
```

```
Depth 제한 없는 의사결정 나무 결과
     0   1
0  350   0
1    0 169
Depth 제한 없는 의사결정 나무 모형 평가
              precision    recall  f1-score   support

           0     1.0000    1.0000    1.0000       350
           1     1.0000    1.0000    1.0000       169

    accuracy                         1.0000       519
   macro avg     1.0000    1.0000    1.0000       519
weighted avg     1.0000    1.0000    1.0000       519
```

```
[53] print("랜덤 포레스트 결과(100 개의 tree 사용)")
pd.DataFrame(confusion_matrix(y_target, pred_RF))

print("랜덤 포레스트 모형 평가")
print(classification_report(y_target, pred_RF,digits=4))
```

```
랜덤 포레스트 결과(100 개의 tree 사용)
     0   1
0  350   0
1    0 169
랜덤 포레스트 모형 평가
              precision    recall  f1-score   support

           0     1.0000    1.0000    1.0000       350
           1     1.0000    1.0000    1.0000       169

    accuracy                         1.0000       519
   macro avg     1.0000    1.0000    1.0000       519
weighted avg     1.0000    1.0000    1.0000       519
```

의사결정 나무
(Depth 제한 없는)

랜덤 포레스트

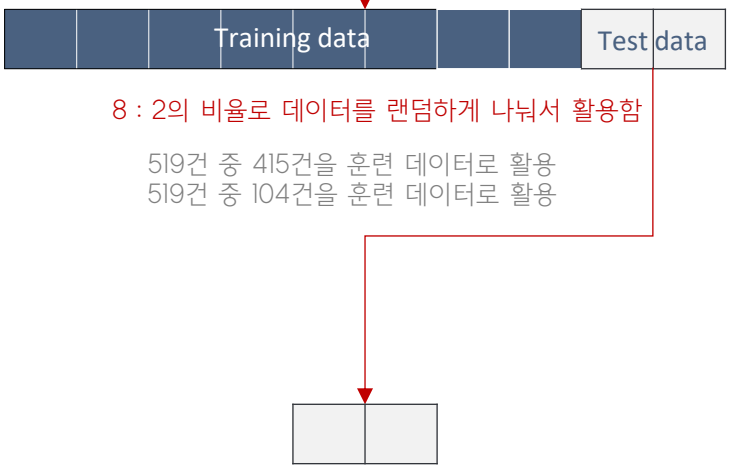
5.3 실습

훈련 데이터와 테스트 데이터의 활용 (8:2 비율 활용)

```
[57] # 훈련 데이터와 테스트 데이터를 8:2의 비율로 나눔
     from sklearn.model_selection import train_test_split

     X_train, X_test, y_train, y_test = train_test_split(X_data, y_target, test_size = 0.2, random_state = 123)
     X_train.shape
     X_test.shape

     (415, 30)(104, 30)
```



8 : 2의 비율로 데이터를 랜덤하게 나눠서 활용함

519건 중 415건을 훈련 데이터로 활용
519건 중 104건을 훈련 데이터로 활용

모형들을 비교하여 최종 모형을 선정하기 위해
Test data를 활용하여 모형을 평가 함

2.1.로지스틱 회귀분석 (Logistic Regression)

```
[61] # 표준화 작업
     scaler = StandardScaler()
     scaler.fit(X_train)
     X_scale_tr = pd.DataFrame(scaler.transform(X_train), columns = X_train.columns)
     X_scale_te = pd.DataFrame(scaler.transform(X_test), columns = X_test.columns)

     # 로지스틱 회귀 모형 생성
     Logistic = LogisticRegression()
     Logistic.fit(X_scale_tr, y_train)

     # 예측 확률 (TEST 데이터로 예측하여 평가)
     probs_te = Logistic.predict_proba(X_scale_te)[:,1]

     # 타겟 변수 예측
     pred_TEST_logistic = np.where(probs_te.reshape(-1) >= 0.5, 1, 0)
```

2.2. 의사 결정 나무(Decision Tree)

```
[62] # 의사결정나무(Decision Tree) 모형 생성
     DTree_train = tree.DecisionTreeClassifier(random_state=300)
     DTree_train.fit(X_train, y_train)

     # 의사결정나무(Decision Tree) 모형 생성 (depth를 3으로 준것)
     DTree3_train = tree.DecisionTreeClassifier(random_state=300, max_depth = 3)
     DTree3_train.fit(X_train, y_train)

     # 타겟 변수 예측 (depth를 주지 않은 것과, 준 것)
     pred_TEST_Tree  = DTree_train.predict(X_test)
     pred_TEST_Tree3 = DTree3_train.predict(X_test)
```

2.3. 랜덤 포레스트(Random Froest)

```
[63] # 100개의 decision tree를 사용한 랜덤 포레스트 (max_features는 sqrt()함수를 사용하는 것이 디폴트임)
     RF_train = RandomForestClassifier(n_estimators=100, random_state=0)
     RF_train.fit(X_train, y_train)

     # 예측값 생성
     pred_TEST_RF = RF_train.predict(X_test)
```

5.3 실습

테스트 데이터를 활용한 모형 평가

```
[69] print("Logistic 결과")
     pd.DataFrame(confusion_matrix(y_test, pred_TEST_logistc))

     print("Logistic 모형 평가")
     print(classification_report(y_test, pred_TEST_logistc,digits=4))
```

```
Logistic 결과
    0  1
0  76  0
1   2 26
Logistic 모형 평가
              precision    recall  f1-score   support

           0     0.9744    1.0000    0.9870        76
           1     1.0000    0.9286    0.9630        28

    accuracy                         0.9808       104
   macro avg     0.9872    0.9643    0.9750       104
weighted avg     0.9813    0.9808    0.9805       104
```

```
[70] print("Depth 제한 없는 의사결정 나무 결과")
     pd.DataFrame(confusion_matrix(y_test, pred_TEST_Tree))

     print("Depth 제한 없는 의사결정 나무 모형 평가")
     print(classification_report(y_test, pred_TEST_Tree,digits=4))
```

```
Depth 제한 없는 의사결정 나무 결과
    0  1
0  73  3
1   4 24
Depth 제한 없는 의사결정 나무 모형 평가
              precision    recall  f1-score   support

           0     0.9481    0.9605    0.9542        76
           1     0.8889    0.8571    0.8727        28

    accuracy                         0.9327       104
   macro avg     0.9185    0.9088    0.9135       104
weighted avg     0.9321    0.9327    0.9323       104
```

```
[71] print("Depth를 3으로 제한한 의사결정 나무 결과")
     pd.DataFrame(confusion_matrix(y_test, pred_TEST_Tree3))

     print("Depth를 3으로 제한한 의사결정 나무 모형 평가")
     print(classification_report(y_test, pred_TEST_Tree3,digits=4))
```

```
Depth를 3으로 제한한 의사결정 나무 결과
    0  1
0  73  3
1   4 24
Depth를 3으로 제한한 의사결정 나무 모형 평가
              precision    recall  f1-score   support

           0     0.9481    0.9605    0.9542        76
           1     0.8889    0.8571    0.8727        28

    accuracy                         0.9327       104
   macro avg     0.9185    0.9088    0.9135       104
weighted avg     0.9321    0.9327    0.9323       104
```

```
[72] print("랜덤 포레스트 결과(100 개의 tree 사용)")
     pd.DataFrame(confusion_matrix(y_test, pred_TEST_RF))

     print("랜덤 포레스트 모형 평가")
     print(classification_report(y_test, pred_TEST_RF,digits=4))
```

```
랜덤 포레스트 결과(100 개의 tree 사용)
    0  1
0  75  1
1   4 24
랜덤 포레스트 모형 평가
              precision    recall  f1-score   support

           0     0.9494    0.9868    0.9677        76
           1     0.9600    0.8571    0.9057        28

    accuracy                         0.9519       104
   macro avg     0.9547    0.9220    0.9367       104
weighted avg     0.9522    0.9519    0.9510       104
```

로지스틱 회귀모형

5.3 실습

최종 결과 확인

```
[76] # 최종 데이터의 X 데이터 추출/ y 데이터 추출
     y_fin  = final_testDF[['diagnosis']]
     X_fin  = final_testDF.drop('diagnosis', axis = 1)

     # 랜덤 포레스트와 의사결정나무 결과 확인
     pred_FIN_RF = RF.predict(X_fin)
     pred_FIN_Tree  = DTree.predict(X_fin)
```

```
[79] # 스케일링
     X_scale_fin = pd.DataFrame(scaler.transform(X_fin), columns = X_fin.columns)

     # 예측 확률 (TEST 데이터로 예측하여 평가)
     probs_fin = Logistic.predict_proba(X_scale_fin)[:,1]

     # 타겟 변수 예측
     pred_FIN_logistc = np.where(probs_fin.reshape(-1) >= 0.5, 1, 0)
```

랜덤 포레스트

```
print("랜덤 포레스트 결과(100 개의 tree 사용)")
pd.DataFrame(confusion_matrix(y_fin, pred_FIN_RF))

print("랜덤 포레스트 모형 평가")
print(classification_report(y_fin, pred_FIN_RF,digits=4))
```

랜덤 포레스트 결과(100 개의 tree 사용)

	0	1
0	7	0
1	4	39

랜덤 포레스트 모형 평가

	precision	recall	f1-score	support
0	0.6364	1.0000	0.7778	7
1	1.0000	0.9070	0.9512	43
accuracy			0.9200	50
macro avg	0.8182	0.9535	0.8645	50
weighted avg	0.9491	0.9200	0.9259	50

의사결정 나무 (Depth 제한 없는)

```
print("Depth 제한 없는 의사결정 나무 결과")
pd.DataFrame(confusion_matrix(y_fin, pred_FIN_Tree))

print("Depth 제한 없는 의사결정 나무 모형 평가")
print(classification_report(y_fin, pred_FIN_Tree,digits=4))
```

Depth 제한 없는 의사결정 나무 결과

	0	1
0	7	0
1	3	40

Depth 제한 없는 의사결정 나무 모형 평가

	precision	recall	f1-score	support
0	0.7000	1.0000	0.8235	7
1	1.0000	0.9302	0.9639	43
accuracy			0.9400	50
macro avg	0.8500	0.9651	0.8937	50
weighted avg	0.9580	0.9400	0.9442	50

로지스틱 회귀모형

```
print("Logistic 결과")
pd.DataFrame(confusion_matrix(y_fin, pred_FIN_logistc))

print("Logistic 모형 평가")
print(classification_report(y_fin, pred_FIN_logistc,digits=4))
```

Logistic 결과

	0	1
0	7	0
1	1	42

Logistic 모형 평가

	precision	recall	f1-score	support
0	0.8750	1.0000	0.9333	7
1	1.0000	0.9767	0.9882	43
accuracy			0.9800	50
macro avg	0.9375	0.9884	0.9608	50
weighted avg	0.9825	0.9800	0.9805	50

최종적으로 결과를 확인해 볼 때 모든 데이터를 활용한 모형은 overfitting 문제로 비교적 예측성능이 떨어졌으나
훈련 데이터와 테스트 데이터를 나눠 사용한 경우 여전히 성능이 높은 것을 알 수 있음